

PROJECT 1

Predicting Laptop Prices Using ML (Major)

This Project will discuss about:

1. Obtaining Laptop Prices dataset
2. Basic data exploration
3. Feature Engineering
4. Explanatory Data Analysis (EDA)
5. Data Pre-processing
6. Modelling

Obtaining Laptop Prices Dataset

For this project, I've obtained my dataset from Kaggle. This dataset contains 1300 rows of data and 12 columns (features) that we could focus on to build our prediction model.

Basic Data Exploration

After loading the dataset via Pandas, we can see a list of laptops and specs that are associated with each laptop. Looking at the dataset, we can see that some columns such as ScreenResolution and Cpu have alphanumeric data while other features consist of purely numerical or alphabetical values. These data would need to be filtered and engineered later.

To avoid any complications and error-prone predictions, useless features such as "Unnamed:0", "Company" and "Product" will be removed from the dataset. (Having an apple laptop with windows OS doesn't seem to make any sense for price prediction.)

We would then check for any missing values that are present in the dataset. We wouldn't want any errors to result from analysis or modelling later.

Feature Engineering

We would now extract and reorganize our data to better understand the underlying factors that contribute to the price of laptops. If we take a look at the ScreenResolution column, there seems to be laptops with touchscreen capabilities. Since touchscreen laptops are known to be more expensive than those without them, a TouchScreen feature would be added to mark laptops with such capabilities. We would then extract and replace the screen resolution column with their respective pixel count using regular expressions. I find regular expressions incredibly useful when it comes to extracting/filtering alphanumeric values.

We would then apply the same process for engineering the Cpu, Ram, and Weight features. Our goal is to minimize or remove any units and words that are not essential for analysis later.

Now comes the most tiring part of feature engineering, dealing with memory feature. Upon closer inspection, the memory column contains various types of memory (SSD, HDD, SSHD, and Flash Storage). We would need to create 4 additional columns representing different memory types and extract their memory capacities individually. (Additional processing needs to be done for laptops having double memory configuration that uses the same memory types.(EX: 256GB SSD + 512GB SSD). This could be done using a similar process shown above.

Having those memory configurations handled, I've decided to drop the GPU column entirely as it contains a high variability of GPUs. Intel GPUs are integrated GPUs, Nvidia GPUs are discrete while AMD GPUs are either integrated or discrete. Labelling and classifying each would take significant effort and time which may or may not contribute to the modelling process later on.

Explanatory Data Analysis (EDA)

Using our feature-engineered dataset, we can now plot graphs and compute tables to visualize how each feature relates to the variability of laptop prices.

By using the `.barplot` method imported from Matplotlib, we can test and verify our hypothesis or initial opinions on how some features will affect the pricing of laptops. Here's an illustration of plotting a barplot for the feature `TypeName` (type of laptop).

From the barplot above, we can rectify and conclude that, on average, workstation and gaming laptops have a higher price than other types of laptops. This is to be expected as these types of laptops often have better spec configurations (better CPU, more memory, etc) to meet the demands of clients in the professional workspace. Notebooks and netbooks have lower prices due to their low-powered configurations. Higher Ram capacities also reflect higher prices in laptops.

Plotting bar graphs on the `Cpu` features shows some interesting results. In general, higher-powered processors should be priced higher than lower-powered ones. The prices for intel processors generally follow this pattern (Xeon > i7 > i5 > i3) and the same principles apply to AMD CPUs as well (Ryzen > AMD A series > E series). The barplot obtained from the dataset shows otherwise.

Data Pre-processing

We will relabel and convert categorical features into numerical features. This is essential for training our ML models as ML models only accept numerical values as inputs.

Knowing that the TouchScreen feature only has 2 categories, we can use label encoding to encode this feature. (One-hot-encoding can be used too) Using the Scikit-learn's label encoding function, the variables present in TouchScreen ('No', 'Yes') will be encoded into 0s and 1s.

Label encoding also handles features with high cardinalities. Applying label encoding to the Cpu feature, the label encoded values (associated with their pre-encoded variables) are recorded for predicting purposes later.

Other features with slightly lower cardinality were encoded via the one-hot-encoding method. Through the use of pandas' .getdummies() method, a new column will be created to indicate the presence of each categorical variable.

After applying One-hot-encoding to TypeName and OpSys features, we will use manual encoding to deal with features with high cardinality if we know the order of variables.

We can use python's dictionary and mapping methods to specify and encode each category based on their magnitude/order. The code snippet shown below encodes the ScreenResolution feature based on the pixel count.

Modelling

After loading the pre-processed .csv dataset, we identify our dependant variable (Price) and allocate a separate data frame for the target variable.

We can then split the dataset for training and validating the performance of the models we are going to apply later. Roughly 30% of the data would be used to test our ML models later.

we begin to train and validate the performance for different models. By applying **linear regression**, we get **80.73% accuracy**. To prevent from overfitting we use **ridge regression** , getting **81.27**. **And by applying** Random Forest getting **88.73%** .The two main metrics used for estimating the performance of our models would be the R-squared score and the Mean absolute error (MAE). In general, we want to achieve a higher R-squared score and lower MAE score with our models.